EBugDec: Detecting Inconsistency Bugs caused by RFC Evolution in Protocol Implementations

Jingting Chen
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security,University of Chinese Academy of Sciences
Beijing, China

Feng Li*
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security,University of Chinese Academy of Sciences
Beijing, China

QingFang Chen
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security,University of Chinese Academy of Sciences
Beijing, China

Ping Li
School of Cyber Security,University of Chinese Academy of Sciences
Beijing, China

Lili Xu
Institute of Information Engineering, Chinese Academy of Sciences
Beijing, China

Wei Huo
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security,University of Chinese Academy of Sciences
Beijing, China

ABSTRACT

The implementation of network protocol must comply with respective Request for Comments (RFC) and updated as RFCs evolve. However, due to the richness of RFCs and the complex relationships between them, systematically discovering the evolution of RFC requirements is non-trivial, which consequently brings in inconsistency bugs when modifying code to support new RFC documents. This can lead to inconsistency bugs when modifying code to support new RFC documents, known as RFC-evolutionary bugs or *ebugs*. Recent approaches have used natural language processing techniques to extract RFC rules and employed differential testing or static analysis to discover inconsistency bugs in protocol implementations. However, they seldom consider the evolution of RFC requirements nor their related bugs.

In this paper, we present EBugDec. Given a protocol implementation and the RFCs it claims to support, our approach identifies evolutionary relationships between RFC documents and their corresponding requirement changes. From this, we derive two major types of evolutionary rules: primitive rules that dictate requirements for newly-introduced packet items, and derivative rules that describe the influence the new items made on requirements stipulated in earlier RFCs. Both of them are represented in formal expressions that dictate packet-related operations should be guarded by specific conditions under special cases (if necessary). Then we use clues found in code annotations and release notes to locate rule-related code in the implementation, and leverage a predominator-based algorithm to discover rule violations in the implementation. We also uncover incomplete error handling logic when the rule-specified conditions fail. We implemented a prototype of EBugDec and demonstrated its efficiency by applying it on 12 implementations of protocol services, along with 178 RFC documents their historical releases claim to support. On average, EBugDec consumed 37.29 seconds to finish its analysis, and detected 17 new *ebugs*, 5 of which can only be triggered under harsh prerequisites.

*corresponding author

KEYWORDS

Network protocol, RFC, Inconsistency bug

ACM Reference Format:
Jingting Chen, Feng Li, QingFang Chen, Ping Li, Lili Xu, and Wei Huo. 2023. EBugDec: Detecting Inconsistency Bugs caused by RFC Evolution in Protocol Implementations. In *The 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '23), October 16–18, 2023, Hong Kong, China.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3607199.3607222

1 INTRODUCTION

The implementations of network protocols must comply with requirements stated in respective Request for Comments (RFC), which are evolving continuously. Over the past 5 years, a total of 29 new RFC standards have been published for the Border Gateway Protocol (BGP). Among them, 25 update or supplement requirements described in earlier RFCs, while the remaining ones obsolete and replace historical RFCs. The evolution of RFC documents result in code changes in the corresponding protocol implementations. Our statistics (c.f. Section3.1) on 6 widely-used protocol implementations show that almost 65% of code submitted to major releases of the implementations is intended to support new RFC requirements, such as parsing and responding to packets with newly-registered fields or parameters.

Previous research [35] has shown that code changes can introduce software bugs. Our statistics in Section 3.1 also show that 67% of the vulnerabilities discovered in the aforementioned protocol implementations are related to RFC evolutionary code changes. Consider the vulnerability CVE-2019-5747 in the DHCP implementation of Busybox. In RFC 2132 [40], all of the vendor information extensions defined for the Bootstrap Protocol (BOOTP) are used as Dynamic Host Configuration Protocol (DHCP) options. Consequently, a series of new requirements are added for the *option* field of DHCP packet introduced in RFC 2131 [27]. One of the new requirements specifies that the field size should be 4 bytes when the type of the field is set to DHCP_SUBNET. However, when adding

support for RFC 2132, the programmers forgot to meet this requirement, leading to an out-of-bound read vulnerability. By exploiting this vulnerability, an attack can obtain sensitive information from the stack.

Bugs like CVE-2019-5747 are the result of inconsistencies between RFC evolution and the intended code changes meant to support it. In this paper, they were referred to as RFC-evolutionary bugs, or *ebugs* for short. The main challenge in detecting *ebugs* is systematically discovering the evolutionary RFC requirements and their violations in protocol implementations, which is non-trivial.

Firstly, the RFC documents of a protocol are numerous and have complex relationships with each other. A newly-published RFC may not only introduce requirements for new packet fields or parameters, but also replace or supplement the original requirements defined in earlier RFCs. Moreover, RFC specifications are written in natural language that might be unstructured. Therefore, manually extracting all RFC evolutionary specifications needs significant human efforts and is error-prone. Recent approaches [20, 43] employ natural language processing techniques to extract rules from RFC documents, and utilize differential testing or static analysis to discover inconsistency bugs in protocol implementations. However, these approaches often do not consider relationships between different RFC documents or the evolution of RFC rules, leading to the omission of *ebugs*.

Second, RFC-evolutionary bugs often lack common code patterns, making them difficult to detect through static approaches without domain knowledge. Besides, features introduced in new-supported RFCs may only be triggered under specific configurations or with certain prerequisites for backward compatibility. Dynamic approaches [36, 43, 50], such as fuzzing and differential testing, may have to exhaustively test all possible combinations of configuration items, package sequences, and parameter values in order to uncover potential bugs in the code, which is time-consuming.

To address these challenges, we propose EBugDec, an approach that focuses on statically and efficiently locating RFC-evolutionary bugs in protocol implementations. Previous work [20] has shown that approximately 76% of known vulnerabilities on popular network protocol can be triggered by hand-crafted network packets. Furthermore, we observe that most RFC evolution pertains to packet-related properties, particularly the decoding and handling logic of a packet. This allow us to concentrate on identifying and detecting two primary types of evolutionary rules on packets, including primitive rules that specify properties for fields or parameters introduced by newly-supported RFCs, and derivative rules that replace or supplement properties specified for existing field or parameters in earlier RFCs. As illustrated in Figure 1a, RFC requirements usually contain domain-specific nouns, such as packet, field or parameter names, as well as comparative and error-handling phrases. The relationships between RFC documents can be obtained from the document heading or IANA information (shown in Figure 1c). Additionally, rules derived from evolved RFC requirements typically correspond to condition check or error-handling code in protocol implementations (shown in Figure 1d, line 23) .

Based on these findings, we utilize domain-specific keywords and heuristic strategies to identify sentences that update or supplement properties of packet items and then apply NLP to extract primitive

... (**Capabilities Optional Parameter**) contains one or more triples:

```
+------------------------+
| Capability Code (1 octet)   |
+------------------------+
| Capability Length (1 octet) |
+------------------------+
| Capability Value (variable) |
+------------------------+
```

... its peer **doesn't support this capability**, the speaker **MAY** send a NOTIFICATION message to the peer and terminate peering. The **Error Subcode** in the NOTIFICATION message is then set to **Unsupported Capability**.

(a) RFC 5492 (Excerpt)

3.1. Long-lived Graceful Restart Capability(Capability code 71):
The **capability value** consists of zero or more tuples as follows:

```
+-----------------------------------+
| Address Family Identifier (16 bits)   |
+-----------------------------------+
| Subsequent Address Family Identifier (8 bits) |
+-----------------------------------+
| Flags for Address Family (8 bits)   |
+-----------------------------------+
| Long-lived Stale Time (24 bits)   |
+-----------------------------------+
```

(b) draft-uttaro-idr-bgp-persistence (Excerpt)

Capabilities Code
Reference: RFC5492

Value	Description	Reference
1	Multiprotocol Extensions for BGP-4	[RFC2858]
69	ADD-PATH Capability	[RFC7911]
71	Long-Lived Graceful Restart (LLGR) Capability	[draft-uttaro-idr-bgp-persistence]

(c) Registery info maintained on IANA (Excerpt)

```
1   struct capability_header { uint8_t code; uint8_t length;};
2   static int bgp_capability_parse(struct peer *peer, size_t
      length){
3     struct stream *s = BGP_INPUT(peer);
4     struct capability_header caphdr;
5     caphdr.code = stream_getc(s);
6     caphdr.length = stream_getc(s);
7     switch (caphdr.code) {
8       case CAPABILITY_CODE_MP:
9         ret = bgp_capability_mp(peer, &caphdr);
10        break;
11      case CAPABILITY_CODE_ADDPATH:
12        ret = bgp_capability_addpath(peer, &caphdr);
13        break;
14  +   case CAPABILITY_CODE_LLGR:
15  +     ret = bgp_capability_llgr(peer, &caphdr);
16  +     break;
17    }
18    if(ret <0) return -1;
19  }
20  + static int bgp_capability_llgr(struct peer *peer, struct
      capability_header *caphdr){
21  /* Violation of derivative rule: chk_bf(Capability_Length % 7 ==
      0, access(Capability_Value)) if Capability_Code == 71 */
22  +  struct stream *s = BGP_INPUT(peer);
23  +  while (caphdr->length >= 4) {
24  +    iana_afi_t pkt_afi = stream_getw(s);
25  +    iana_safi_t pkt_safi = stream_getc(s);
26  +    uint8_t flags = stream_getc(s);
27  +    uint32_t stale_time = stream_get3(s);
28  +  }
29  +  return 0;
30  +}
31  uint32_t stream_get3(struct stream *s){
32      if (STREAM_READABLE(s) < 3) {assert(0); return 0;}
33      uint32_t l = s->data[s->getp++] << 16;
34      l |= s->data[s->getp++] << 8;
35      l |= s->data[s->getp++];
36      return l;
37  }
```

(d) Code Snippet in FRRouting 8.4.2

Figure 1: Examples of evolutionary rules

rules from them. To further derive how property changes of a particular item affect its upper structure, we reconstruct hierarchical packet structure based on RFC relationships to help infer derivative rules. Both primitive rules and derivative rules are expressed in formal expressions (as shown in Figure 1d, line 21), which dictate packet-related operations should be guarded by specific conditions under special cases (if necessary).

Our approach aims to locate violations for evolutionary bugs in source code implementations. To achieve this goal, a technical problem that need to be addressed is to identify code that should have been evolved in accordance with the extracted evolutionary rules. Previous work [20, 32] has focused on locating data structures or parsing functions based on the length and offset specified for each field in the RFC. However, evolutionary rules are more likely to be implemented as optional fields or configurable functions in protocol implementations to maintain backward compatibility. The offset of an optional field is variable, and it may share the same format with other optional fields, making it difficult to locate. Our key intuition is that programmers tend to declare newly-supported RFCs and their corresponding packet items in release notes or code annotations of the implementations. We leverage this information to identify code that accesses these items and check if conditions protecting the code comply with those specified in the evolutionary using a predominator-based algorithm. We also track the error codes set on the false branch of the conditions to ensure they are handled properly. Both violations of rule-specified conditions and incomplete error handling are reported as *ebugs*.

We implemented a prototype of EBugDec and demonstrated its efficiency by applying it on 12 implementations of protocol services, along with 178 RFC documents that they claim to support in historical releases. On average, EBugDec consumes 37.29 seconds to finish its analysis. Through our evaluation, We detected 17 new evolutionary bugs, among which 5 can only be triggered under specific configuration or harsh prerequisites.

The contributions of our approach are summarized as follow:

(1) We unveil interesting findings about RFC evolution and present a rule extractor to automatically obtain primitive rules for packet items introduced by new RFC, as well as derivative rules the new items made on existing RFC requirements.
(2) We design EBugDec, a static analysis approach that identifies locations in source code implementation that should be updated with the extracted rules, and detects both rule condition violations and incomplete error handling issues.
(3) We implement a prototype of EBugDec which supports C/C++ protocol implementations and apply it on 12 application subjects, resulting in the discovery of 17 new RFC evolution bugs. 6 of these bugs have already been confirmed and fixed by developers.

2 PRELIMINARIES AND DEFINITIONS

RFC Requirements. Requirements on protocol implementations are typically defined in Request For Comments (RFCs) through natural language and graphical structures. These requirements guide developers in implementing the protocol correctly and ensuring security. Prior work [20] observed that requirements described in RFCs often follow specific semantic structures that can be (semi-)automatically extracted and represented as formal rules. Recent approaches [18, 20, 43] have utilized RFC rules to improve the efficacy of locating inconsistency bugs or identifying semantic-gap attacks. However, these approaches usually do not consider relationships between RFC documents, which may lead to the omission of important rules describing the evolution of RFC requirements.

Relationships between RFC Documents. As stated in RFC 2223 [37], it is common for an RFC to provide additional information on a topic that has been previously discussed in a previous RFC or completely replaces an earlier RFC. These relationships are referred to as "Updates" and "Obsoletes" between RFC documents. One can easily obtain these two types of relationships as it is stipulated in RFC 2223 that if an RFC updates or obsoletes another RFC, this information is indicated as third line on the left side of the heading. A document that obsoletes an earlier document can stand on its own, indicating that all RFC rules extracted from the former document will replace those extracted from the latter. However, a document that updates an earlier RFC cannot stand on its own. This means rules extracted from the former document would replace or supplement those describing the same topic in the earlier RFCs.

Unlike updating, the supplementing relationships are not always explicitly declared in the RFC documents. According to RFC 8126 [22], protocols often use different constant values to identify various protocol parameters, which helps to promote the extensibility and interoperability. Values assigned to newly-supported protocol parameters are coordinated by the Internet Assigned Numbers Authority (IANA), as shown in Figure 1c, to ensure there is no conflicting use. Documents that register new constant values and their corresponding parameters for an existing packet field also describe conditions under which the new values should be used or the structure of the new parameters.

RFC Evolutionary Rules. The evolutionary relationships between RFC documents, especially updating and supplementing new parameters for existing packet fields, can lead to the evolution of RFC requirements. These evolutionary changes are governed by rules known as Evolutionary Rules, which fall into two categories: 1) primitive rules defining properties for new topics (e.g. new parameters, fields or packets) introduced in a newly-supported RFC document, and 2) derivative rules that enhance or clarify properties specified for existing topics in earlier RFCs.

Figure 1a shows the original structure of *Capabilities Optional Parameter* as described in RFC 5492. A typical supplementation made on the *Capabilities Optional Parameter* field by registering a new *Capability Code (71)* with corresponding structure of *Capability Value* is illustrated in Figure 1b. Figure 1c provides the registry information maintained by IANA. The supplementation clarifies the length of the field when the *Capability Code* is 71, leading to a derivative rule shown on line 21 of Figure 1d. Code changes made to support the supplementation violate the rule in FRRouting 8.4.2, as depicted in Figure 1d.

3 OVERVIEW

3.1 A Motivating Example

We have investigated the code changes made in 6 protocol implementations of BGP and DHCP. For each implementation, we

queried all of its historical release notes using regular expression "support/add.*RFC" and manually review the submitted code to confirm if the RFCs returned by the search were implemented. According to our statistics (shown in Figure 2), 65% of historical code changes in a new release of the under-considering implementations were made to support new RFC requirements, with 98% of these new requirements related to packet parsing and processing. We also searched for bug fixing commits and found that 67% of them involved RFC-related code changes.

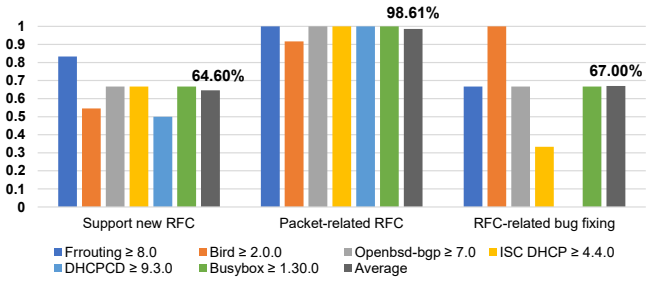


Figure 2: Statistics on RFC-related Code Changes in 6 Protocol Implementations

Figure 1 illustrates an example of a vulnerability resulting from inconsistencies in a programmer's code implementation for RFC evolution. The vulnerability exists in the BGP implementation of FRRouting 8.4.2 known as CVE-2023-31489. Successful exploitation can cause an out-of-bound read at line 27, which can potentially leak sensitive information or even result in a Denial of Service (DoS) attack. The principle of this vulnerability is as follows.

RFC 4271 [47], the standard RFC of BGPv4, describes the basic structure of a BGP packet, which has been updated and supplemented over time by newly introduced RFC documents. One such document is RFC 5492 [42], which introduces an optional parameter called *Capabilities* that allows for graceful capability advertisement without terminating BGP peering. As shown in Figure 1a, the *Capabilities* field contains at least one TLV triple, and the last item in a triple is a variable-length field named *Capability Value*. In addition to RFC 5492, the BGP implementation also support an Internet draft named draft-uttaro-idr-bgp-persistence [19], as stated in the implementation's release notes. This draft supplements the requirements defined for *Capabilities*. Specifically, the draft registers new types and their corresponding *Capability Codes* and make refinements to the *Capability Value* field when the new type codes are used. Figure 1b shows that *Capability Code* 71 is assigned to a new type of *Capabilities* named *Long-lived Graceful Restart Capability*(LLGR). When this *Capability Code* is used, the *Capability Value* field defined in RFC 5492 is no longer a variable-length field. Instead, it consisted of fix-length triples, each of them has a length of 7 bytes (16+8+8+24 bits/8). Requirement change for the *Capabilities* field can be represented in formal expression *chk_bf(Capability_Length % 7 == 0, access(Capability_Value)) if Capability_Code == 71*, which means that when the *Capability Code* is LLGR, the *Capability Value* field as well as its sub-fields can only be accessed and processed if the length of *Capabilities* is no less than 7 bytes. However, in the case of FRRouting 8.4.2, when adding support for draft-uttaro-idr-bgp-persistence, it did not follow the evolutionary design for the Capability Code of LLGR type. The vulnerable code snippet is shown in Figure 1d. Line 3 extracts the received packet from *peer* into *s*. Lines 4-6 parse the corresponding fields from *s* and store them in the structure. Lines 7-17 call different parsing functions for the Capability Value based on the type of the Capability Code. In the case of LLGR type, the *bgp_capability_llgr* function is called in line 15. This function checks the length of the Capability Value in the received packet (line 23), but the length check is limited to 4 bytes instead of the 7 bytes specified in the draft. If a maliciously crafted LLGR Capability Value field with a length between 5 and 6 bytes is constructed, it will result in reading an additional 1-2 bytes from s at line 27, ultimately leading to a Denial of Service at line 32.

As previously mentioned, discovering such a vulnerability is non-trivial. There are 87 RFC documents that update or supplement RFC 4271. However, only 7 documents (8%) explicitly declare their relationships with RFC 4271. Therefore, it is necessary to identify implicit supplementation, such as the RFC descriptions shown in Figure 1b, in order to systematically reconstruct the relationships between RFC documents supported by an implementation and derive evolutionary rules. This is particularly important for rules related to existing fields, such as the aforementioned requirements for the *Capabilities* field under the LLGR type. Identifying code related to new requirements introduced by RFC evolution can be challenging due to the backward compatibility of the protocol. Newly-introduced items, such as the *Capabilities* field, are often defined as optional fields that contain a series of Type-length-value(TLV) triples. In FRRouting 8.4.2, for example, there are 8 similar data structures related to the *Capabilities* field. It is difficult for a static analyzer to differentiate codes accessing these different optional fields during rule violation detection.

3.2 Our approach

As we can see, inconsistency bugs can arise when programmers misunderstand or incompletely implement evolutionary rules while modifying code to support new RFC requirements. In this paper, we focus on how to extract or derive RFC evolutionary rules and discover their violations (i.e.,*ebugs*) in source code implementations. Given a protocol implementation, a workflow of EBugDec is illustrated in Figure 3, which consists of three major steps.

Step 1: The Evolutionary Rule Extractor identifies updating and supplementing relationships between RFC documents with the help of information maintained on IANA. Packet-related meta-info are also extracted from each document to help recover a hierarchical structure of packet together with the evolutionary relationships. The Extractor then traverses the packet structure in a bottom-up order to generate primitive rules for each updating or supplementing field. It also derives the impact of the evolution on the original design of the upper-layer structure in order to infer derivative rules.

Step 2: Using the packet structure recovered in the first step, the Rule-Related Code Identifier maps the packet fields checked or accessed in the evolutionary rules to the corresponding parsing functions in the implementation. For similar structures, their parsing functions and processing codes are identified based on text information, including code annotations and release notes.

Step 3: The Rule Violation Detector checks for inconsistencies between evolutionary rules and their corresponding code locations.

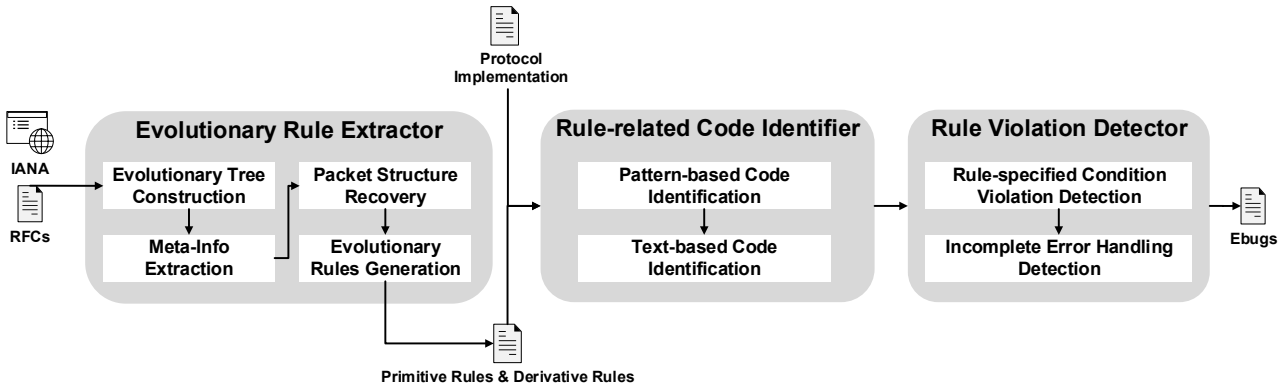


Figure 3: Architecture of EBugDec

To accomplish this, it uses a predominator-based algorithm to detect whether the code locations are guarded by the rule-specified conditions. It then traces the error code set on the false branch of each condition and check whether a complete error handling logic has been implemented. Inconsistency bugs are reported if rule violations or incomplete error handling paths are found.

We illustrated how our approach works using the example in Figure 1. First, EBugDec obtains the implicit supplementing relationship between draft-uttaro-idr-bgp-persistence and RFC 5492 based on the constant values and their corresponding parameter types maintained in IANA (as shown in Figure 1c). We then extracts meta-info from each document, including rules specified for the *Capabilities* field in RFC 5492 and those defining the newly introduced sub-fields of *Capability Value* under different *Capability Code* (shown in Figure 4). Using this information, we reconstruct a hierarchy packet structure, as depicted in Figure 5. During the bottom-up traversal of the structure, EBugDec extracts primitive rules for the sub-fields of *Capability Value* from the draft's meta-info (Rule 3 in Figure 5) and derives derivative rules that capture the influence of the sub-fields on the value range and length of *Capabilities* (Rule 4&5 in Figure 5). To locate the rule-related code changes, we search the FRRouting release notes and find that *LLGR* has been supported since version 8.2.2. By examining the code changes committed to this historical version, we identify that modifications made to data structure *s* in line 24 and to the function *bgp_capability_parse* in line 15 in Figure 1d are related to the field. For the code snippet shown in Figure 1d line 23, the length check performed for the *Capabilities* field before parsing its *LLGR* capability value does not comply with the condition specified in Rule 4, our approach reports an ebug. This inconsistency bug has already been confirmed by developers as a vulnerability in the latest version of FRRouting.

4 TECHNICAL DETAILS

We designed and implemented EBugDec through three sub-analysis (Figure 3): Evolutionary Rule Extractor, Rule-Related Code Identifier and Rule Violation Detector.

4.1 Evolutionary Rule Extractor

The Evolutionary Rule Extractor takes RFC documents that the protocol implementation claims to support as input and reconstructs a hierarchical packet structure based on the relationships between RFC documents and meta-info extracted from each document. It then traverses the packet structure in a bottom-up order to extract the attributes of newly-introduced fields or parameters and their impacts on the original design of the upper packet structures. This process results in two types of evolutionary rules: primitive rules and derivative rules.

Constructing RFC Evolutionary Tree. As mentioned earlier, our approach addresses two types of evolution between RFC documents: updating and supplementing. To identify the updated document for a given RFC document RFCi, we look for the "Updated by ..." description in the heading of RFCi. Supplementing relationships, on the other hand, are not explicitly defined in the documents. We extract them by crawling and analyzing the Protocol Registry maintained by IANA. This registry contains information about the newly-introduced packet items, including their names, value ranges and RFC documents that define them, as shown in Figure 1c.

The extracted updating and supplementing relationships are represented in the form of tuples, denoted by *<org_rfc, new_rfc, rel_type, org_item, new_attr>*. Here, org_rfc and new_rfc represent the original and new documents that are related through an evolution, respectively, rel_type indicates whether the evolutionary relationship is updating or supplementing, org_item represents the packet item in org_rfc that is updated or supplemented with a property new_attr, by new_rfc. For example, from the IANA registry depicted in Figure1c, we can obtain the following relationship between RFC 5492 and draft-ttaro-idr-bgp-persistence: *<RFC 5492, draft-ttaro-idr-bgp-persistence, Supplement, Capabilities Code, Long-lived Graceful Restart(LLGR) Capability>*. These tuples form an RFC evolution tree with the oldest RFC document that the implementation supports as the root.

Extracting Packet-related Meta-info from each RFC. In order to recover a hierarchical packet structure that the implementation accepts and deduce evolutionary rules based on it, we extract packet-related meta-info for each RFC document on the evolutionary tree.

```
{
  "Capabilities": {
    "refer": "RFC 5492"
    "struct": {
      "Capability Code": {
        "bitwidth":8,
        "vaild_range":{
          "Reserved": { "value": "0" }
        },
        "Orig_Rules":[chk_bf( range(Capability_Code)=[0],
    access(Capability_Code)), chk_bf(!range(Capability_Code)=[0],
    retError(Error_Subcode, Unsupported_Capability))]
      },
      "Capability Length": {
        "bitwidth": 8,
        "Orig_Rules": [chk_bf(Capability_Length ==
    sizeof(Capability_Value), access(Capability_Length))]
      },
      "Capability Value": {
        "bitwidth": 8
      }
    }
  }
}
```

Figure 4: An Example of Meta-info

The meta-info includes information such as the name, offset and size of each packet field or parameter. Such information is usually described in structured forms, such as diagrams or pseudocode, and can be easily extracted. For example, we can obtain the size of three packet fields from the graphical description in Figure 1a. However, more comprehensive meta-info, such as the associations with other fields or corresponding codes when errors occur, needs to be extracted from natural language sentences.

Previous work [18] has shown that when an RFC describes the attributes of a field or its relationship with other fields, it typically uses modal keywords (such as "MUST," "SHOULD") in accordance with RFC 2119 [17] or includes comparative keywords (such as "is identical to") in the narrative. We identify sentences that contain both the packet field name and modal/comparative keywords as the target sentences, and use NLP technology to extract rules in the form of chk_bf (cond, op). These rules are then stored in meta-info (as shown in Figure 4). For RFC documents with the evolutionary type of "updating", phrases such as "new text" and "replace with" are often used to replace paragraphs in the old document. Therefore, the scope of identifying target sentences and extracting rules can be further narrowed down to these paragraphs.

Generating Evolutionary Rules based on Packet Reconstruction. Having obtained the RFC evolutionary tree and meta-info of each document, we proceed to traverse the tree in a bottom-up manner to reconstruct the packet structure accepted by the implementation. This allows us to capture the impact of new packet fields or parameters on themselves and the upper-layer structures, generating both primitive rules and derivative rules.

It should be noted that field or parameter names may differ between IANA and RFC, as both are manually written and maintained. As a result, it may be difficult to find the corresponding *org_item* and *new_attr* in the meta-info as recorded on the evolutionary tree. Taking BGP as an example, only 50% of the 243 field or parameter names registered on IANA can be found in meta-info if exact string matching is used. To improve the matching accuracy for the remaining fields and parameters, a heuristic strategy is adopted. This strategy involves considering the order of appearance of the fields or parameters in comparison to other fields at the same hierarchical packet structure level in the RFC, as well as their valid value ranges.

Primitive rules related to new_attr can be obtained from the meta-info of the corresponding new_rfc based on its evolutionary type, after matching with the IANA registry. For items with an evolutionary type of "updating", we record the original rules related to new_attr in the meta-info of new_rfc as primitive rules. For items with an evolutionary type of "supplementing", we generate primitive rules with if conditions based on the attribute records of new_attr in the meta-info under different values. For example, Rule 3 in Figure 5 is a primitive rule extracted based on the meta-info of draft-ttaro-idr-bgp-persistence, which specifies the structure of *Capability Value* under the condition of *Capability_Code == 71*.

To obtain derivative rules, we first integrate the new_attr structure into the corresponding offset of $orig_item$ based on the relationship in the evolution tree. We then analyze the impact of new_attr on the meta-info of $orig_item$. Typically, fields that can be supplemented to support diverse features are designed as a triple of <type, length, value>, known as TLV. To support a new feature, such as transmission of new type of information, one only needs to add a new type to the field and specifies the corresponding length and structure of value. However, the new feature can affect the original rules of the field stored in relevant meta-info, resulting in derivative rules as follow, where *DR* is short for derivative rule and *OR* is short for original rule:

$$DR_{len}(fld) = \begin{cases} chk_bf(fld.len == valid_len, op(fld)), & if\ type == ty \\ OR_{len}(fld), & otherwise \end{cases} \quad (1)$$

$$DR_{range}(fld) = chk_bf(valid_range\ union\{ty\}, op(fld)) \quad where\ OR_{range}(fld).cond == valid_range \quad (2)$$

Both Rules 4 and 5 shown in Figure 5 are derivative rules that describe the impact on the size and valid type code range of *Capabilities* caused by the supplement of Capability code in the draft.

4.2 Rule-Related Code Identifier

Using the extracted evolutionary rules as input, Rule-Related Code Identifier maps packet fields appear in the rules to data structures and code that accesses these structures in the protocol implementation. This is achieved by inducing code parsing patterns and using hints from text information such as code annotations and release notes. This provides a basis for detecting inconsistency bugs.

Identifying Rule-related Code with Parsing Patterns. Previous work [20, 32] primarily relied on identifying code with the same patterns of packet/field structure to locate parsing functions, followed by using data flow analysis to find code that accesses the parsed fields. For example, in Figure 1d, the parsing code implemented for *Capability_Header* can be located based on its subfield sequence and the offset of each subfield in lines 5-6. However, fields involved in derivative rules often use the TLV structure, as previously mentioned, resulting in similar code patterns of data structures or parsing functions, such as the *bgp_capability_mp* and *bgp_capability_addpath* functions depicted in Figure 6. Developers may also use a unified parsing function to handle fields with the same structure, like the *parse_option_buffer* function, which is responsible for parsing all different types of option fields in the

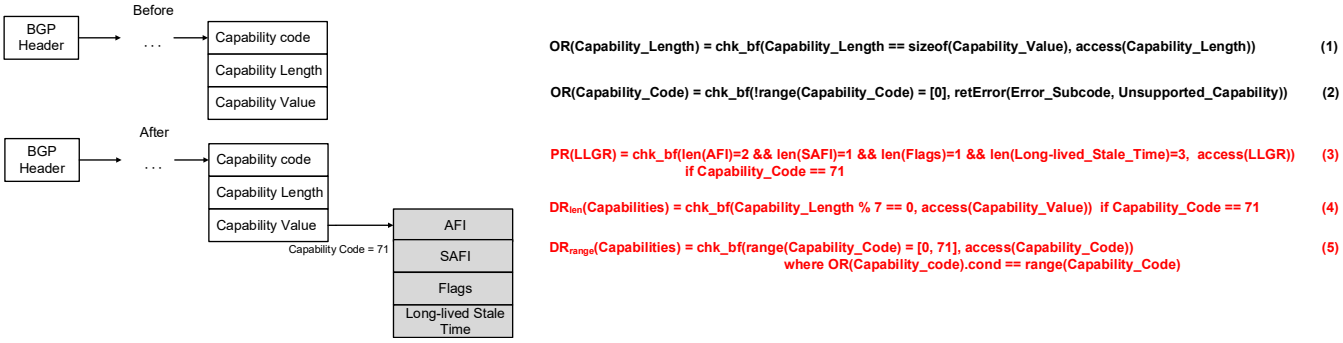


Figure 5: An Example of Evolutionary Rule Extraction

FreeBSD implementation of DHCP protocol. This make it difficult to locate the function used to parse a specific field, and thus, challenging to identify the corresponding code that accesses the field.

Identifying Rule-related Code with Text Info. Based on our analysis, we have found that text sources such as code annotations and release notes contain valuable information that can help differentiate data structures or parsing code corresponding to fields with similar structures. Table 1 summarizes the mapping relationship between packet fields and text information for 7 protocol implementations. We obtained these statistics by extracting code annotations and sentences from release notes containing keywords such as "Support", "Add", and "RFC", identifying RFC numbers and supported/added feature names, and cross-referencing this information with the meta-info extracted from relevant RFC documents supported by the implementation. Our results indicate that approximately 82% of packet fields can be identified with the help of text sources.

Table 1: Statistics on relationships between packet fields and text information in 7 protocol implementations

Implementations	#RFC	#Comment	#Release	#Total	%
FRRouting 8.4.2	30	22	20	23	76.7%
Bird 2.0.12	26	19	13	19	73.1%
OpenBSD 7.2-BGP	18	13	15	16	88.9%
ISC DHCP 4.4.3	18	10	8	12	66.%7
BusyBox 1.36.0	10	10	0	10	100%
FreeBSD 13	4	4	-	4	100%
OpenBSD-DHCP 7.2	7	5	10	5	71.4%
Total/AVG	113	83	77	89	82.4%

Based on the aforementioned findings, we propose a three-step method to use code annotations and release notes to distinguish the access positions of similar packet fields in protocol implementations. Given a series of packet fields *flds*, the parsing function list *parseList(fld)* and access position list *posList(fld)* obtained for each field through their structures and parsing patterns, we refine the results of pattern-based rule-related code identification as follows:

Firstly, for each *pos* in *posList(fld)*, we check if the code comment at the corresponding position includes the name of *fld*. If it does, we record it as a location that accesses *fld* and add the *pos* and *parseList(fld)* to the refined list *refined(fld)*. For example, in Figure 6, we can determine from the comments that the fields named *afi* and *safi* are accessed in line 20 and the parsing function for their upper-level structure *addpath* in the hierarchical packet the can be inferred accordingly.

Secondly, for fields that appear in the evolutionary rules but have an empty *refined(fld)* list after the first step, we indirectly infer their access positions based on fields whose refined list is non-empty and share the same parsing function list with *fld*. For instance, in Figure 6, although there is no comment in line 9 where *mp.afi* is accessed, we can inferred that *bgp_capability_mp* is the parsing function of *mp* since the function has similar code pattern with *bgp_capability_addpath* but is called when the *Capability Code* is *CAPABILITY_CODE_MP*. Then, we use data flow analysis to find code that accesses the parsed fields and add line 9 in Figure 1d into the refined list.

Finally, for fields that still have an empty refined list after the first two steps, we can use release notes to indirectly infer possible code modifications to support the fields or their corresponding RFCs. Take LLGR introduced by draft-uttaro-idr-bgp-persistence as an example. We found by searching the release notes that FRRouting has supported LLGR since version 8.2.2. From the commit code of version 8.2.2, we can identified the updated data structure, as shown in Figure 1. Functions in *ParseList(LLGR)* that access the updated structure are considered to be the parsing function and added to *refined(LLGR)*.

4.3 Rule Violation Detector

To detect rule violations, the Rule Violation Detector takes the rule-related code identified in the previous step as input and performs consistency checks between RFC rules and the corresponding code implementation. The detector checks whether the specified conditions (cond) in the rules are executed before performing the operations (op), and whether appropriate error handling logic is in place when the conditions are not met.

Detecting Violations of Rule-specified Conditions. Given an evolutionary rule, we aim to detect whether the conditions specified in the rule are checked on each possible path from the entry to the code location (*loc*) corresponding to an RFC-specified operation in the protocol implementation. To make the detection process more efficient, we modify and adopt the predominator-based rule-violation algorithm proposed by [20].

```
1  static int bgp_capability_mp(struct peer *peer, struct
     capability_header *hdr){
2    if (hdr->length != 4)
3      return -1;
4
5    mpc->afi = stream_getw(s);
6    mpc->reserved = stream_getc(s);
7    mpc->safi = stream_getc(s);
8
9    peer->afc_recv[mpc->afi][mpc->safi] = 1;
10   return 0;
11 }
12 static int bgp_capability_addpath(struct peer *peer,  struct
     capability_header *hdr){
13   if (hdr->length % 4) {
14     return -1;
15   iana_afi_t pkt_afi = stream_getw(s);
16   iana_safi_t pkt_safi = stream_getc(s);
17   uint8_t send_receive = stream_getc(s);
18
19   /* Convert AFI, SAFI to internal values, check. */
20   bgp_map_afi_safi_iana2int(pkt_afi, pkt_safi, &afi, &safi);
21
22   return 0;
23 }
24 void bgp_process_packet(struct thread){
25   struct peer *peer;
26   peer = THERAD_ARG(thread);
27
28   // indirect call
29   ret = bgp_capability_parse(peer, opt_length);
30   if ret < 0:
31     bgp_stop(peer);
32 }
```

Figure 6: An Example of Rule-related Field Identification

We start by collecting constraints on the control flow graph (CFG) of the function where *loc* is located, from the predominators of *loc* to *loc*, and denote them as *cond(loc)*. We then check if the conjunction of *cond(loc)* and the negation of conditions in the evolutionary rule is satisfiable. If it is, then there is at least one path in the implementation that does not comply with the rule, and a consistency bug should be reported at *loc*. If no violation is found, we proceed by analyzing the caller functions starting from the current function. For each direct caller function, we collect the constraints from the predominators of the callsite of the current function (referred to as *cs*) to *cs*. These constraints are then solved in conjunction with the negation of *cond(loc)*. If at least one caller function yields a satisfiable solution, it indicates a rule violation. Otherwise, we continue this process for the caller functions until a rule violation is detected or until we reach the outermost parsing function of the packet structure. Although this algorithm has the potential to result in an exponential growth in the size of the function call chain to be explored and the constraints to be solved, in practice, we have observed that analyzing up to 6 layers of caller functions is sufficient to terminate the analysis.

For the vulnerability shown in Figure 1, the path constraints generated at line 23 are represented as $Capability_Length <= 4$. This condition, when combined with the negation of condition given in Figure 5 Rule 4, becomes satisfiable, leading to denial of service.

Detecting Incomplete Error Handling Logic. When receiving illegal or malicious network packages, it is crucial to perform proper error handling to prevent further functional or security issues. Figure 6 illustrates an example where if the condition check in line 2 fails, the function sets the return value to -1, indicating that an error has been occurred. This error code is then propagated through subsequent function return values until it reaches the BGP packet parsing function, *bgp_process_packet*. Finally, the error is passed to the *bgp_stop* function, which performs necessary clean up operations, such as clearing the error and resetting the established connection.

To identify incomplete error handling logic, we need to verify whether an error code is set when the condition check specified in the evolutionary rule fails, and whether the error code is passed to the error handling function. In Figure 1, although the length of the *Capability_Length* field is checked in line 23 under the LLGR condition, no error code is set for when the check fails, resulting in malicious input not being detected and stopped in a timely manner.

We have observed that error codes are often represented as constant values less than 0 and do not participate in arithmetic operations. Furthermore, the error handling function is typically responsible for preventing the further transmission of illegal or malicious packet by closing the connection. Thus, we propose a method for detecting incomplete error handling issues by first collecting the return values that appear on the false branch of rule-specified conditions and tracing their accessed locations in the caller functions until reaching the outermost packet parsing function. Values that participate in arithmetic operations can be disregarded. If the collected return values are overwritten by values greater than 0 or are not passed to the upper-level caller function, an incomplete error handling issue should be reported.

5 EVALUATION

We implemented EBugDec based on NLTK [2], LLVM [1] and z3 solver [6]. Our implementation is about 3.2 KLOC of Python code and 3.5 KLOC of C++ code. In this section, we introduce our experimental setup and provide an evaluation of effectiveness and efficiency of EBugDec. We also present case studies of new *ebugs* found by EBugDec.

5.1 Setup

We applied EBugDec to 12 implementations of 4 fundamental network protocols: Border Gateway Protocol (BGP), Dynamic Host Configuration Protocol (DHCP), Network Time Protocol (NTP), and Transport Layer Security version 1.3 (TLS1.3). BGP is an inter-Autonomous System routing protocol that enables the exchange of network reachability information between BGP-speaking systems. DHCP provides a framework for passing configuration information to hosts on a TCP/IP network. NTP is a protocol that helps synchronize the clock times of computers in a network. TLS 1.3 is a cryptographic protocol designed to provide communications security over a computer network.

We obtained a total of 12 implementations of these protocols shown in Table 2. For our demonstration, we selected the latest versions of these implementations. Two of the selected implementations, FRRouting [10] and Bird [7], are routing protocol suites implemented in C/C++ for Linux and Unix platform. Both of the protocol suites support BGP, and they are widely used in data centers

Table 2: Twelve Protocol Implementations under Testing

Protocol	Implementation	#Supported RFCs
BGP [47]	FRRouting 8.4.2	80
	Bird 2.0.12	43
	OpenBSD 7.2	38
DHCP [27]	ISC DHCP 4.4.3	24
	DHCPCD 9.4.1	45
	BusyBox 1.35.0	14
	FreeBSD 13	18
	OpenBSD7.2	24
NTP [23]	NTPSec 1.2.1	11
	NTP 4.2.8	7
	OpenBSD 7.2	3
TLS1.3 [28]	Wolfssl-5.4.0	14

and academic/research community. OpenBSD[12] is a UNIX-like operating system that develops its own BGP, DHCP and NTP implementations. In addition, We selected two implementations for DHCP, namely ISC DHCP[11] and DHCPCD[9], as well as two others implemented in BusyBox[8] and FreeBSD[13]. For NTP, we selected NTPSec[15] and NTP[14]. Finally, we selected Wolfssl[16], which is a lightweight, portable, C-language-based SSL/TLS library targeted at IoT, embedded, and RTOS environments.

Table 2 also provides an overview of the number of RFC documents claimed to be supported by each protocol implementation in their respective release notes and code comments (Column *#Supported RFCs*). To collect the relevant documents, we searched for keywords "RFC" and "draft" in the implementations. It is evident from the table that there are significant variations in the number of RFC documents supported by different implementations of the same protocol. During our demonstration, our focus is solely on identifying discrepancies between the implementation and the RFC documents that the implementation explicitly declares to support. By doing so, we aim to minimize the occurrence of false positives in the detection of rule violations, particularly for optional features that the implementation does not actually support. Note that, for Wolfssl, documents that describe certificates and encryption/decryption algorithms are beyond the scope of this experiment.

All of our evaluations are running on an Intel(R) Xeon(R) Gold 5218 processor with 64GB of memory.

5.2 Effectiveness

Evolutionary Rules Extracted from RFCs. Table 3 statistics evolutionary rules extracted from the RFC documents supported by each implementation shown in Table 2. Overall, we have extracted 759 evolutionary rules from a total of 178 RFC documents across 12 implementations. Among these rules, 83.4% are derivative rules, while the rest are primitive rules. These rules involve a total of 542 packet fields, and their corresponding accessed code locations must be identified in the relevant implementation to detect *ebugs*. These findings highlight the importance of considering the relationships between RFCs, especially the impact that new RFCs have on earlier design, when extracting rules. Prior approaches [18, 20, 43, 50] that extract rules from each RFC document separately may overlook a significant proportion of evolutionary rules.

Table 4 investigates the influence of new-introduced fields or parameters on their upper-level packet structure for BGP, DHCP and NTP protocols by considering all their valid RFC documents.

Table 3: Statistics of Rules Extracted from RFCs

Implementation	#Evol. Rules	#Prim. Rules	#Deri. Rules	#fld
FRRouting-BGP	**62**	16	46	24
Bird-BGP	**47**	9	38	17
OpenBSD-BGP	**37**	6	31	11
ISC DHCP-DHCP	**124**	13	111	87
DHCPCD-DHCP	**151**	32	119	119
BusyBox-DHCP	**98**	10	88	88
FreeBSD-DHCP	**82**	5	77	87
OpenBSD-DHCP	**97**	8	89	76
NTPSec-NTP	**5**	2	3	3
NTP-NTP	**5**	2	3	3
OpenBSD-NTP	**5**	2	3	3
Wolfssl-TLS1.3	46	21	25	24
Total / %	**759**	16.6%	83.4%	542

Column *#RFC* displays the total number of valid RFC documents for each protocol. TLS is excluded from this analysis as we did not analyze its certificate and encryption/decryption related RFC documents. Column *#Fields* represents the number of packet fields whose properties are influenced by RFC evolution, while column *#RC* indicates the number of times the field has been changed during its evolution. Our statistics show that, on average, requirements specified for each packet field may be changed 5.68 times by newly published RFC documents. The impact can be classified into two categories: changes made on the valid value range, which accounts for an average of 16.6% (Column *Range*), and those made to the length that takes up 83.4% (Column *Length*), of the upper-levels structure. The result also indicates the importance and universality of derivative rules.

Table 4: Frequency and Impact of Field Changes in Three Network Protocols

Protocol	#RFC	Total		Range		Length	
		#Fields	#RC	#Fields	#RC	#Fields	#RC
BGP	84	19	124	15	71	4	53
DHCP	69	9	75	4	6	5	69
NTP	11	8	11	5	6	3	5
Total	178	36	210	24	83	12	127
Avg/%.	59.33	12	5.83	8	39.52%	4	60.48%

Code Identified for Rule Detection. Table 5 presents a comparison between the pattern-based code identifying method (Column *Pattern Only*), which only uses the similarity between parsing function code pattern and the packet structure to identify rule-related code, and the pattern-based and text-based method (Column *Pattern+Text*), which additionally uses clues from code annotations and release notes to identify rule-related code.

As shown in the table, the former method can only identify 18.12% (*#Func*/(*#fld-UImpl*)) of the fields involved in the evolutionary rules that each protocol implementation needs to support (Table 3, Column *#fld*). Other fields have not been found their corresponding parsing functions (Column *#Func*) and access locations (Column *#Loc*) in the corresponding implementation. Additionally, there is a false negative rate of 83.94%(*#FN*/*#fld*) for fields with similar structures.

However, by adding clues extracted from code annotations and release notes, the number of identified fields with corresponding parsing functions and access locations increases to 95.97%, with a

lower false positive (3.36%) and false negative (4.03%). False positives are present in ISC DHCP because the option fields of DHCPv4 and DHCPv6 share the same structure without comments or committed code in the implementation to differentiate them. False negatives exist in the three NTP implementations since they have limited RFC-related comments, rendering the text-based method less effective.

Column *UImpl* presents the numbers of fields or parameters each protocol implementation claims to support but that are actually unimplemented. For example, in the case of the three BGP implementations, none of them has implemented a type code named *Multiple Labels Capability* for the *Capability Optional Parameter* in the OPEN package of BGP, despite all three claiming to support RFC 8277 [39] where *Multiple Labels Capability* is registered and used to enable the capability to bind MPLS Labels to Address Prefixes in BGP. Similar situations occur in the DHCP implementations being considered. RFC 2132 defined 76 new DHCP option types, each with a specific format. Since the optional parameters defined in RFC allow developers to use their discretion, developers may make different choices regarding which requirements they would like to implement, leading to discrepancies between what is claimed to be supported and what is actually implemented. EBugDec generates warning messages to alert developers of potential functionality or interoperability issues that may arise due to unimplemented packet fields or parameters. By addressing these warnings, developers can ensure that their code is compliant with the relevant RFCs and will work correctly in a wide range of environments.

Table 5: Comparison of Code Identifying Methods

Implementation	Pattern Only			Pattern+Text			UImpl
	#Func	#Loc	FP/FN	#Func	#Loc	FP/FN	
FRRouting-BGP	7	9	8/9	23	53	0/0	1
Bird-BGP	3	3	3/11	16	46	0/0	1
OpenBSD-BGP	1	1	4/6	10	36	0/0	1
ISC DHCP-DHCP	2	4	37/50	15	50	5/0	72
DHCPCD-DHCP	2	16	1/118	19	45	0/0	100
BusyBox-DHCP	2	4	0/88	7	16	0/0	81
FreeBSD-DHCP	2	6	4/83	12	22	0/0	75
OpenBSD-DHCP	2	7	4/72	14	20	0/0	62
NTPSec-NTP	1	3	0/2	1	3	0/2	0
NTP-NTP	1	3	0/2	1	3	0/2	0
OpenBSD-NTP	1	3	0/2	1	3	0/2	0
Wolfssl-TLS1.3	3	7	14/7	24	46	0/0	0
Total/ %	18.12%	66	67/455	95.97%	343	5/6	393

Ebugs Found in Latest Implementations. Table 6 summarizes the results of EBugDec in discovering new *ebugs* in the 12 protocol implementations. As shown in the table, EBugDec has generated a total of 23 bug reports, 6 of them are false positives. Of the remaining 17 new bugs, 13 are results of rule-specified condition violations, while the other 4 are due to incomplete error handling logic. We manually examine the 13 rule violations and discovered that 7 of them can be exploited to cause Denial of Service (DoS), while the others may lead to out-of-bound reads and potentially leak sensitive packet information into log files. There are also 3 new bugs related to the error handling logic, all of which are present in Wolfssl 5.4.0. Currently, 6 out of the total bugs we found have been confirmed and addressed by the developers.

False positives were detected in the implementation of FRRouting 8.4.2 for BGP and ISC DHCP 4.4.3. In the case of BGP, the implementation stores the length of the packet into an array. However, due to the limitations of static analysis, EBugDec incorrectly identified the code accessing a rule-related field, resulting in a false positive in rule violation detection. Regarding ISC DHCP, EBugDec encounters a challenge in distinguishing between the parsing functions for the option fields of DHCPv4 and DHCPv6 packets, as mentioned earlier. The issue arises from mistakenly identifying the option type of DHCPv6 as an option type of DHCPv4 with the same value, despite their distinct functionalities and different option value lengths. A notable example is the *MTU Interface* option in DHCPv4 and the *IAPREFIX* option in DHCPv6, which both share an assigned value of 26.This misidentification leads to the generation of false positives during the rule violation detection process. This particular false positive does not occur when detecting vulnerabilities in other protocol implementations. The reason for this is that these implementations have well-designed modular structures, which helps us to accurately identify the code related to each specific version.

Table 6: New Bugs Detected by EBugDec

Implementation	#Reports	#New Bugs		#FP
		#Cond	#Error	
FRRouting-BGP	8	7	0	1
Bird-BGP	0	0	0	0
OpenBSD-BGP	1	1	0	0
ISC DHCP-DHCP	10	5	0	5
DHCPCD-DHCP	0	0	0	0
BusyBox-DHCP	0	0	0	0
FreeBSD-DHCP	1	0	1	0
OpenBSD-DHCP	0	0	0	0
NTPSec-NTP	0	0	0	0
NTP-NTP	0	0	0	0
OpenBSD-NTP	0	0	0	0
Wolfssl-TLS1.3	3	0	3	0
Total	23	13	4	6

Validating Existing Vulnerabilities. We conducted an evaluation of EBugDec by gathering vulnerabilities reported over the past five years. To select the vulnerabilities, we searched the CVE database using keywords such as "packet," "message," and "options." We focused on vulnerabilities that were triggered due to evolutionary rule-related issues. As a result, we identified 8 vulnerabilities for further analysis, including 3 related to BGP and 5 related to DHCP, show in Table 7.

EBugDec has successfully detected 7 out of the 8 vulnerabilities during the evaluation. In the case of the remaining vulnerability in DHCPCD, the implementation does perform a check on the length field of the option before using the value field. However, the correctness of the *length* parameter used in the *memcpy* function is not verified, which falls outside the scope of EBugDec's analysis. To effectively detect this particular vulnerability, it is necessary to incorporate traditional Buffer-Overflow detection methods that analyze the lengths of both the source (*src*) and destination (*dst*) parameters within the *memcpy* function. Additionally, we came across 1 false positive during the detection of FRRouting 8.4. This occurred as we failed to recognize the presence of a length field stored in an array, as mentioned earlier when evaluating bugs in the latest implementations.

Table 7: Existing Vulnerabilities Benchmark and the result of EbugDec

Protocol	CVE ID	Implementation	Rule	Found	False Positive
BGP	CVE-2022-40302	FRRouting 8.4	RFC 9072 Sec2 Fig2 Extended Parameters Format	✓	1
	CVE-2022-43681	FRRouting 8.4	RFC 9072 Sec2 Fig1 Extended Encoding OPEN Format	✓	
	CVE-2022-40318	FRRouting 8.4	RFC 9072 Sec2 Fig1 Extended Encoding OPEN Format	✓	
DHCP	CVE-2019-5747	BusyBox 1.30.0	RFC 2132 Sec3.3 Subnet Mask	✓	0
	CVE-2018-20679	BusyBox 1.29.3	RFC 2132 Sec3.3 Subnet Mask	✓	0
	CVE-2019-11579	DHCPCD 7.2.0	RFC 2132 Sec9.3 Option Overload	✓	0
	CVE-2019-11766	DHCPCD 7.2.0	RFC 6603 Sec4.2 Prefix Exclude Option	✓	
	CVE-2019-11577	DHCPCD 7.2.0	RFC 8415 Sec21.6 IA Address Option	×	

5.3 Efficiency

Table 8 presents the time and memory usage of EBugDec in detecting *ebugs* for the 12 implementations. Column *Step.1* displays the time spent on extracting evolutionary rules, which does not include the time required for crawling RFC documents and information on IANA since it is considered as a one-time job. Column *Step.2* provides the time for identifying parsing functions and accessing code locations for fields in the evolutionary rules, including the time spend on searching for rule-related clues in comments, release notes, and committed codes. Column *Step.3* gives the time for detecting rule violations and incorrect error handling issues for each implementation. Columns *Time* and *Mem* summarize the total time and the maximum memory usage, respectively, for detecting *ebugs* for each implementation. For protocol implementations ranging in size from 4.6 to 924 KLOC, EBugDec takes an average of 37.29 seconds to complete its analysis with a maximum memory usage of 0.3 GB maximum memory.

Table 8: Time (second) and Memory (GB) Usage of EBugDec

Implementation	#LOC	Step.1	Step.2	Step.3	Total	Mem
FRRouting-BGP	131K	66.31	21.09	5.00	92.40	0.22
Bird-BGP	6.2K	27.26	3.22	2.39	32.97	0.13
OpenBSD-BGP	39K	10.27	8.11	1.59	19.97	0.06
ISC DHCP-DHCP	10.1K	8.05	47.15	10.35	65.55	0.30
DHCPCD-DHCP	34K	14.06	52.84	1.34	68.24	0.05
BusyBox-DHCP	7.8k	5.04	28.74	2.04	35.82	0.16
FreeBSD-DHCP	6.8k	8.03	16.38	1.01	25.42	0.03
OpenBSD-DHCP	8.2K	10.03	14.88	0.46	25.37	0.03
NTPSec-NTP	55K	2.17	2.5	0.15	2.40	0.03
NTP-NTP	73K	2.17	4.17	0.92	4.00	0.09
OpenBSD-NTP	4.6K	2.19	0.21	0.16	2.42	0.03
Wolfssl-TLS1.3	924k	4.20	1.58	61.50	67.28	0.08
AVG	108.3K	13.31	16.74	7.24	37.29	0.08

5.4 Case Studies of New Bugs

We now provide a detailed description of the newly discovered RFC evolutionary bugs(i.e.,*ebug*) detected by EBugDec. Due to space constraints, we illustrate three typical *ebugs*, including two violations of primitive rule and derivative rule, respectively, and an instance of incomplete error handling.

A Violation of Primitive Rule in ISC DHCP. EBugDec has found a primitive rule violation in the latest DHCP implementation of ISC DHCP 4.4.3. The violation is related to the addition of a new property named *Server Identifier* with type code 54 to the *Option* field of DHCP, as specified in RFC 2132. The RFC also mandates that when the option type is *Server Identifier*, the corresponding *Address* field should be 4 bytes in width, as show in Figure 7a. EBugDec has extracted a primitive rule for the newly-introduced type of Option, as follow: *chk_bf(len(Address)= 4, access(Server_Identifier)) if Option.Type == 54.* However, the code intended to support this new property in ISC DHCP, as shown in Figure 7b dose not do any check. Line 2 and line 4 only search the Server Identifier and read the option into the *data.data* variable, respectively. Line 6 does not check the length of *Address* which stores in *data.data* either, resulting in an out-of-bound memory access. This bug can cause a sensitive information leak of 4 bytes in the log printed by line 7.

9.7. Server Identifier
The identifier is the IP address of the selected server. The code for this option is 54, and its length is 4.

Code	Len	Address			
54	4	a1	a2	a3	a4

(a) RFC 2132 (Excerpt)

```
1  void dhcprequest (packet, ms_nulltp, ip_lease){
2    oc = lookup_option (&dhcp_universe, packet->options,
     DHO_DHCP_SERVER_IDENTIFIER);
3    /* A primtivie rule violation of chk_bf(len(Address)= 4,
     access(Server_Identifier)) if Option.Type == 54. */
4    if (oc && evaluate_option_cache(&data, packet->options, oc ){
5      sip.len = 4;
6      memcpy (sip.iabuf, data.data, 4);
7      sprintf(smbuf, "(%s)", piaddr(sip));
8  }
```

(b) Code snippet in ISC DHCP 4.4.3

Figure 7: An Ebug Detected in ISC DHCP 4.4.3

A Violation of Derivative Rule in FRRouting-BGP. EBugDec has identified a derivative rule violation in the BGP implementation of FRRouting 8.4.2. The violation is related to the impact that a new type of *BGP Prefix-SID* called *SRv6 Service TLVs*, which was introduced by RFC 9252 [30], on the original property of the *BGP Prefix-SID* attribute. This attribute is specified as a TLV format in RFC 8669 [41], and the size of its value field is variable, as shown in Figure 8a. RFC 9252 also mandates that there must have a 1-byte *RESERVED* field and a variable *SRv6 Service Sub TLVs* field in the *Value* field of the *BGP Prefix SID Attribute* when *Type* is 5.

EBugDec has extracted a derivative rule for the *BGP Prefix-SID* attribute field from the supplementation made on its subfields, as follows:*chk_bf(BGP_Prefix-SID.Length >= 1, access(BGP_Prefix-SID.Value)) if BGP_Prefix-SID.Type == 5.* However, the code implemented in FRRouting, shown in Figure 8c, forgot to add the rule-specified condition check before parsing the RESERVED field of this new type of the *BGP Prefix-SID* attribute in line 8. This bug can cause a out-of-bound read and due to there is an *assert* in *stream_getc* function, it will finally cause a Denial of Service.

Triggering this bug requires specific conditions, including a particular packet sequence (i.e. Open, KeepAlive, Update) and the presence of certain Path Attribute Type and BGP Prefix-SID Type in the last packet. As a result, it is less likely to be detected through fuzzing. This bug has been assigned as CVE-2023-31490.

3. BGP Prefix-SID Attribute
The BGP Prefix-SID attribute is defined here to be a set of elements encoded as **"Type/Length/Value"** tuples (i.e., a set of TLVs). All BGP Prefix-SID attribute TLVs will start with a 1-octet type and a 2-octet length.

(a) RFC 8669 (Excerpt)

2. SRv6 Services TLVs (***from RFC 9252***)
The following depicts the SRv6 Service TLVs encoded in the BGP Prefix-SID attribute:

```
0                   1                   2                   3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
| TLV Type  |          TLV Length           | RESERVED |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                SRv6 Service Sub-TLVs                      //
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

(b) RFC 9252 (Excerpt)

```
1  void bgp_attr_psid_sub(uint8_t type, uint16_t length, struct
     peer *const peer){
2    if (type == BGP_PREFIX_SID_SRV6_L3_SERVICE) {
3    /* A derivative rule violation of chk_bf(BGP_Prefix-SID.Length
       >= 1, access(BGP_Prefix-SID.Value)) if BGP_Prefix-SID.Type ==
       5 */
4      if (STREAM_READABLE(peer->curr) < length) {
5        return bgp_attr_malformed(BGP_NOTIFY_UPDATE_ATTR_LENG_ERR);
6      }
7      /* ignore reserved */
8      stream_getc(peer->curr);
9      return bgp_attr_srv6_service(args);
10   }
```

(c) Code snippet in Frrouting 8.4.2

Figure 8: An Ebug Detected in Frrouting 8.4.2

An Incomplete Error Handling in FreeBSD-DHCP. EBugDec have uncovered an incomplete error handling issue in the DHCP implementation of FreeBSD 13. According to RFC 2132, the length of three types of DHPC *Option*, namely Lease Time, Renewal Time, and Rebinding Time, should be 4 bit-width each. The code implemented in FreeBSD to support these new *Option* types, shown in Figure 9, performs a length checking for each type of *Option* in function *pretty_print_option*, line 14. The function returns an error in line 15 if the length check fails. However, the caller function, *check_option*, does not handle the error properly and returns a non-error value without checking the *opbuf* variable returned by *pretty_print_option*. This can result in illegal or even malicious input being accepted without being detected and stopped in a timely manner.

5.5 Limitations

EBugDec has some limitations that should be taken into consideration.

First, it relies on RFC descriptions to identify inconsistencies in protocol implementations. Therefore, it may not be able to detect bugs or vulnerabilities in codes that do not have relevant RFC descriptions. It's important to note that vulnerabilities like CVE-2019-11577, which do not originate from rule violations, also fall outside

```
1  int check_option(struct client_lease *l, int option){
2
3      (opbuf) = pretty_print_option(option, options[option].data,
4              options[option].len, 0, 0);
5     switch (option) {
6         case DHO_DHCP_LEASE_TIME:
7         case DHO_DHCP_RENEWAL_TIME:
8         case DHO_DHCP_REBINDING_TIME:
9              return (1);
10    }
11 }
12 const char * pretty_print_option(unsigned int code, unsigned
     char *data, int len){
13     hunksize = dhcp_options[code].format[i]
14     if (hunksize > len) {
15         return ("<error>");
16         }
17 }
```

Figure 9: An Ebug Detected in FreeBSD 13

the scope of EBugDec's analysis. One can incorporate EBugDec with traditional vulnerability detection approaches to enhance the overall vulnerability detection capability.

Second, EBugDec heavily depends on the quality of code annotations and the standardization of version management. If the code annotations or version management is not standardized, it may lead to difficulties in locating rule-related field parsing functions and accessing locations in the source code, as demonstrated in the case of the three NTP implementations we analyzed. For protocol implementations that lack proper code annotations and standardized version management, additional sources of information such as log messages and variable names can be utilized to aid in inference. Variable names in the code often resemble the corresponding field names in the protocol specification. Similarly, log messages provide insights into the progress and potential errors encountered during message processing. Leveraging these supplementary sources of information can compensate for the lack of standardized annotations and version management, enabling a more accurate analysis by EBugDec.

Finally, EBugDec currently only handles RFC rules related to packet decoding and processing. Future work will focus on dealing with more complex RFC rules and rules specified in other types of specification documents.

6 RELATED WORK

In this section, we give a brief summary of previous work closely related to EBugDec.

Protocol analysis. Rules extracted from RFC have been widely-used by recent approaches to improve the efficacy of detecting protocol implementation bugs. RFCcert [18] extracts rules of certificates according to the RFC standards of SSL/TLS, including updating rules, and uses the obtained certificates to generate test cases and perform differential testing in order to finding bugs in SSL/TLS implementations. Sage [48] presents an approach to automatically generate codes for ICMP based on rules extracted from its RFC. HDiff [43] employs natural language processing techniques to extract rules from RFC documents and utilized deferential testing to discover semantic gap attacks in HTTP implementations. RIB-Detector [20] extracts three types of packet-related rules into a

uniform format and designs a general algorithm to statically detect their violations in implementations. However, neither of these approaches has considered evolution of RFC requirements or their corresponding inconsistency bugs. In this paper, we focus on extracting evolutionary rules and discover their violations in order to find inconsistency bugs introduced by code changes that support new RFCs.

It is worth noting that, unlike traditional network protocols, cellular network protocols describe their specifications in 3GPP documents. BaseSpec[32] analyzes baseband firmware to identify the message decoder and extracts embedded protocol structures in the firmware. It then compares the extracted structures with the specifications defined by 3GPP syntactically and semantically, and reports mismatches. Comparing with RFC documents, 3GPP organizes specifications into different releases, each with distinguishable network capabilities and features [21]. Before a release is frozen, all modifications to its specifications need to be made through a change request, and thus are easier to track compared to RFC evolution. In future work, we also plan to extend EBugDec by dealing with specifications beyond RFCs and protocol implementations without source code.

Protocol Fuzzing. There has been a considerable amount of research on the topic of fuzzing protocol implementations [3–5, 24–26, 29, 36, 38, 44–46, 49]. TCP-fuzz [50] relies on dependencies between packets, systems calls and the state transitions extracted from RFC to guide fuzzing and detect semantic bugs in TCP implementations. SCALE [31] detects RFC compliance errors in DNS nameserver implementations by generating zone files and corresponding queries according to the RFC. AFLNet [36] uses response codes as state-feedback to guide fuzzing and detect stateful bugs in FTP and RSTP protocols. Moreover, DTLS [29] applies state fuzzing to detect the state machine bugs of DTLS protocol implementations by using modal learning to derive the state machine of each implementation and manually comparing it with the RFC specification. While these dynamic approaches have proven effective, EBugDec stands out in its ability to pinpoint the source code locations of evolutionary rule violations, especially those that may only be triggered under specific configurations or harsh prerequisites.

Bugs introduced by Code changes. Several approaches have also been proposed to discover code changes that introduce bugs. HATARI [34] extracts properties from known bugs or their fixing code to identify bug-introducing changes, but it depends strongly on the quality of bug-introducing changes. Mondal [35] focuses on bugs resulting from inconsistent changes in code clones, by extracting the evolutionary history of the clone fragments in open-source software systems, and using clone-types, former clone evolutionary patterns, and clone proximity to find inconsistent changes that are likely to introduce bugs. Another technique, Change Classification [33], trains a classifier using features extracted from the revision history of a software project to determine whether a new software change is more similar to prior buggy changes or clean changes. However, approaches that do not consider domain knowledge, such as those solely relying on code patterns, are limited in locating code changes caused by RFC evolution, which do not share common bug-proneness code patterns.

7 CONCLUSION

We present EBugDec, an approach that focuses on discovering RFC evolution and inconsistencies during code changes in protocol implementations. EBugDec achieves efficiency and effectiveness by identifying evolutionary rules that describe the requirements of newly-introduced packet items, as well as their impact on the original designs specified in earlier RFC documents. The rule-related code implementations are located with clues found in code annotations and release notes. EBugDec then detects and reports both rule-specified condition violations and incomplete error handling logic as *ebugs*. Our evaluation shows that EBugDec can complete its analysis in an average of 37.29 seconds for 12 protocol implementations and has discovered 17 new evolutionary bugs, 6 of which have been confirmed by developers. EBugDec is available at https://github.com/melissa-cjt/EBugDec-main .

ACKNOWLEDGMENTS

We thank anonymous reviewers and shepherd for their comprehensive feedback. This research was partly supported by the National Key Research and Development Program of China(2020YFB1005704), the Strategic Priority Research Program of the CAS(XDC02030200), the Chinese National Natural Science Foundation(62032010, 62202462), the Key Laboratory of Network Assessment Technology, Chinese Academy of Sciences, and Beijing Key Laboratory of Network security and Protection Technology.

REFERENCES

[1] 2009. The LLVM Compiler Infrastructure. https://llvm.org/.
[2] 2009. Natural Language Processing with Python. http://www.nltk.org/.
[3] 2020. beSTORM Black Box Testing. https://beyondsecurity.com/solutions/bestorm.html.
[4] 2020. boofuzz: Network Protocol Fuzzing for Humans. https://boofuzz.readthedocs.io/en/latest/.
[5] 2020. PEACH FUZZER. https://www.peach.tech/.
[6] 2020. The Z3 Theorem Prover. https://github.com/Z3Prover/z3.
[7] 2021. The BIRD Internet Routing Daemon. https://bird.network.cz/.
[8] 2021. BusyBox. https://www.busybox.net/.
[9] 2021. DHCPCD. https://roy.marples.name/projects/dhcpcd/.
[10] 2021. The FRRouting Protocol Suite. https://frrouting.org/.
[11] 2021. Internet Systems Consortium(ISC) DHCP. https://www.isc.org/dhcp/.
[12] 2021. The OpenBSD project produces a FREE, multi-platform 4.4BSD-based UNIX-like operating system. http://www.openbsd.org/.
[13] 2022. freebsd. https://www.freebsd.org/.
[14] 2022. ntp. https://github.com/ntp-project/ntp.
[15] 2022. NTPSec. https://www.ntpsec.org/.
[16] 2022. wolfssl. https://www.wolfssl.com/.
[17] S. Bradner. 1997. Key words for use in RFCs to Indicate Requirement Levels. https://www.rfc-editor.org/rfc/rfc2119/.
[18] Chu Chen, Cong Tian, Zhenhua Duan, and Liang Zhao. 2018. RFC-Directed Differential Testing of Certificate Validation in SSL/TLS Implementations. In *Proceedings of the 40th International Conference on Software Engineering (ICSE '18)*. Association for Computing Machinery, 859–870. https://doi.org/10.1145/3180155.3180226
[19] E. Chen. 2019. Support for Long-lived BGP Graceful Restart. https://datatracker.ietf.org/doc/draft-uttaro-idr-bgp-persistence/.
[20] Jingting Chen, Feng Li, Mingjie Xu, Jianhua Zhou, and Wei Huo. 2022. RIB-Detector: an RFC-guided Inconsistency Bug Detecting Approach for Protocol Implementations. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 641–651. https://doi.org/10.1109/SANER53432.2022.00081
[21] Yi Chen, Di Tang, Yepeng Yao, Mingming Zha, XiaoFeng Wang, Xiaozhong Liu, Haixu Tang, and Dongfang Zhao. 2022. Seeing the Forest for the Trees: Understanding Security Hazards in the 3GPP Ecosystem through Intelligent Analysis on Change Requests. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 17–34. https://www.usenix.org/conference/usenixsecurity22/presentation/chen-yi

[22] M. Cotton. 2017. Guidelines for Writing an IANA Considerations Section in RFCs. https://www.rfc-editor.org/rfc/rfc8126.

[23] J.Martin D. Mills, U. Delaware. 2010. Network Time Protocol Version 4: Protocol and Algorithms Specification. https://www.rfc-editor.org/rfc/rfc5905.

[24] L. Daniel, E. Poll, and J. de Ruiter. 2018. Inferring OpenVPN State Machines Using Protocol State Fuzzing. In *2018 IEEE European Symposium on Security and Privacy Workshops (EuroS PW '18)*. 11–19. https://doi.org/10.1109/EuroSPW.2018.00009

[25] Joeri de Ruiter and Erik Poll. 2015. Protocol State Fuzzing of TLS Implementations. In *24th USENIX Security Symposium (USENIX Security '15)*. USENIX Association, Washington, D.C., 193–206. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/de-ruiter

[26] Joeri de Ruiter and Erik Poll. 2015. Protocol State Fuzzing of TLS Implementations. In *24th USENIX Security Symposium (USENIX Security '15)*. USENIX Association, 193–206. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/de-ruiter

[27] R. Droms. 1997. Dynamic Host Configuration Protocol. https://www.rfc-editor.org/rfc/rfc2131/.

[28] Mozilla E. Rescorla. 2018. Network Time Protocol Version 4: Protocol and Algorithms Specification. https://www.rfc-editor.org/rfc/rfc8446.

[29] Paul Fiterau-Brostean, Bengt Jonsson, Robert Merget, Joeri de Ruiter, Konstantinos Sagonas, and Juraj Somorovsky. 2020. Analysis of DTLS Implementations Using Protocol State Fuzzing. In *29th USENIX Security Symposium (USENIX Security '20)*. USENIX Association, 2523–2540. https://www.usenix.org/conference/usenixsecurity20/presentation/fiterau-brostean

[30] R. Raszuk B. Decraene S. Zhuang G. Dawra, K. Talaulikar. 2022. BGP Overlay Services Based on Segment Routing over IPv6 (SRv6). https://www.rfc-editor.org/rfc/rfc9252.html.

[31] Siva Kesava Reddy Kakarla, Ryan Beckett, Todd D. Millstein, and George Varghese. 2022. SCALE: Automatically Finding RFC Compliance Bugs in DNS Nameservers. In *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022, Renton, WA, USA, April 4-6, 2022*, Amar Phanishayee and Vyas Sekar (Eds.). USENIX Association, 307–323. https://www.usenix.org/conference/nsdi22/presentation/kakarla

[32] Eunsoo Kim, D. Kim, CheolJun Park, Insu Yun, and Y. Kim. 2021. BASESPEC: Comparative Analysis of Baseband Software and Cellular Specifications for L3 Protocols. In *Network and Distributed System Security Symposium(NDSS '21)*.

[33] Sunghun Kim, E. James Whitehead, and Yi Zhang. 2008. Classifying Software Changes: Clean or Buggy? *IEEE Transactions on Software Engineering* 34, 2 (2008), 181–196. https://doi.org/10.1109/TSE.2007.70773

[34] Sunghun Kim, Thomas Zimmermann, Kai Pan, and E. James Jr. Whitehead. 2006. Automatic Identification of Bug-Introducing Changes. In *21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*. 81–90. https://doi.org/10.1109/ASE.2006.23

[35] Manishankar Mondal, Chanchal K. Roy, and Kevin A. Schneider. 2020. A Fine-Grained Analysis on the Inconsistent Changes in Code Clones. In *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 220–231. https://doi.org/10.1109/ICSME46990.2020.00030

[36] V. T. Pham, M. Böhme, and A. Roychoudhury. 2020. AFLNET: A Greybox Fuzzer for Network Protocols. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST '20)*. 460–465. https://doi.org/10.1109/ICST46399.2020.00062

[37] J. Postel. 1997. Instructions to RFC Authors. https://www.rfc-editor.org/rfc/rfc2223.

[38] Van-Thuan Pham Roberto Natella. 2021. ProFuzzBench: A Benchmark for Stateful Protocol Fuzzing. https://arxiv.org/abs/2101.05102

[39] E. Rosen. 2017. Capabilities Advertisement with BGP-4. https://www.rfc-editor.org/rfc/rfc8277/.

[40] R. Droms S. Alexander. 1997. DHCP Options and BOOTP Vendor Extensions. https://www.rfc-editor.org/rfc/rfc2131/.

[41] A. Lindem A. Sreekantiah H. Gredler S. Previdi, C. Filsfils. 2019. Segment Routing Prefix Segment Identifier Extensions for BGP. https://www.rfc-editor.org/rfc/rfc8669.html.

[42] J. Scudder. 2009. Capabilities Advertisement with BGP-4. https://www.rfc-editor.org/rfc/rfc5492/.

[43] Kaiwen Shen, Jianyu Lu, Yaru Yang, Jianjun Chen, Mingming Zhang, Haixin Duan, Jia Zhang, and Xiaofeng Zheng. 2022. HDiff: A Semi-automatic Framework for Discovering Semantic Gap Attack in HTTP Implementations. In *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 1–13. https://doi.org/10.1109/DSN53405.2022.00014

[44] Suphannee Sivakorn, George Argyros, Kexin Pei, Angelos D Keromytis, and Suman Jana. 2017. HVLearn: Automated black-box analysis of hostname verification in SSL/TLS implementations. In *2017 IEEE Symposium on Security and Privacy (SP '17)*. IEEE, 521–538.

[45] Juraj Somorovsky. 2016. Systematic Fuzzing and Testing of TLS Libraries. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. Association for Computing Machinery, 1492–1504. https://doi.org/10.1145/2976749.2978411

[46] M. Tappler, B. K. Aichernig, and R. Bloem. 2017. Model-Based Testing IoT Communication via Active Automata Learning. In *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST '17)*. 276–287. https://doi.org/10.1109/ICST.2017.32

[47] T. Li Y. Rekhter and S. Hares. 2006. A Border Gateway Protocol 4 (BGP-4). https://www.rfc-editor.org/rfc/rfc4271/.

[48] Jane Yen, Tamás Lévai, Qinyuan Ye, Xiang Ren, Ramesh Govindan, and Barath Raghavan. 2021. Semi-Automated Protocol Disambiguation and Code Generation. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) *(SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 272–286. https://doi.org/10.1145/3452296.3472910

[49] H. Zhao, Z. Li, H. Wei, J. Shi, and Y. Huang. 2019. SeqFuzzer: An Industrial Protocol Fuzzing Framework from a Deep Learning Perspective. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST '19)*. 59–67. https://doi.org/10.1109/ICST.2019.00016

[50] Yong-Hao Zou, Jia-Ju Bai, Jielong Zhou, Jianfeng Tan, Chenggang Qin, and Shi-Min Hu. 2021. TCP-Fuzz: Detecting Memory and Semantic Bugs in TCP Stacks with Fuzzing. In *2021 USENIX Annual Technical Conference, USENIX ATC 2021, July 14-16, 2021*, Irina Calciu and Geoff Kuenning (Eds.). USENIX Association, 489–502. https://www.usenix.org/conference/atc21/presentation/zou